RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering



B.Tech. 2nd Year

Object Oriented Methodologies (CS224)

By

Dr. Kalka Dubey

Object Oriented Methodologies (CS224)
11 (3-0-2)

Syllabus

UNIT 1: Fundamental concepts of object oriented programming:

UNIT 2: Classes & Objects

UNIT 3: Inheritance & Polymorphism:

UNIT 4: Standard Template Library (STL) of C++

UNIT 5: Packages & Exception Handling

Objective:

To study the concept of object-oriented programming and practices of C++ programs that leverage the object-oriented methodology features.

Text and Referred Books

Text books:

1. Object Oriented Programming With C++ by E. Balagurusamy, McGraw-Hill Education (India) Pvt.

2. Object-oriented programming in C++ by Lafore, R. Waite Group.

Reference books:

1. Functional and Object-Oriented Analysis and Design by Peretz Shoval, IGI Publishing.

2. Structured and Object-Oriented Analysis and Design Methodology by Pankaj Ghemawat, Pearson Education.

Outcome of the Course

The following are the official course goals agreed upon by the faculty for this course:

- *Concepts of classes & objects and their significance use in the real world.*
- *Benefits of object oriented design and methodology*
- *Build C++ classes using appropriate encapsulation and design principles.*
- *Analyse the utilization of inheritance and polymorphism in the solution of problems.*
- *Choose appropriate object orient programming concepts for solving real-world problems.*

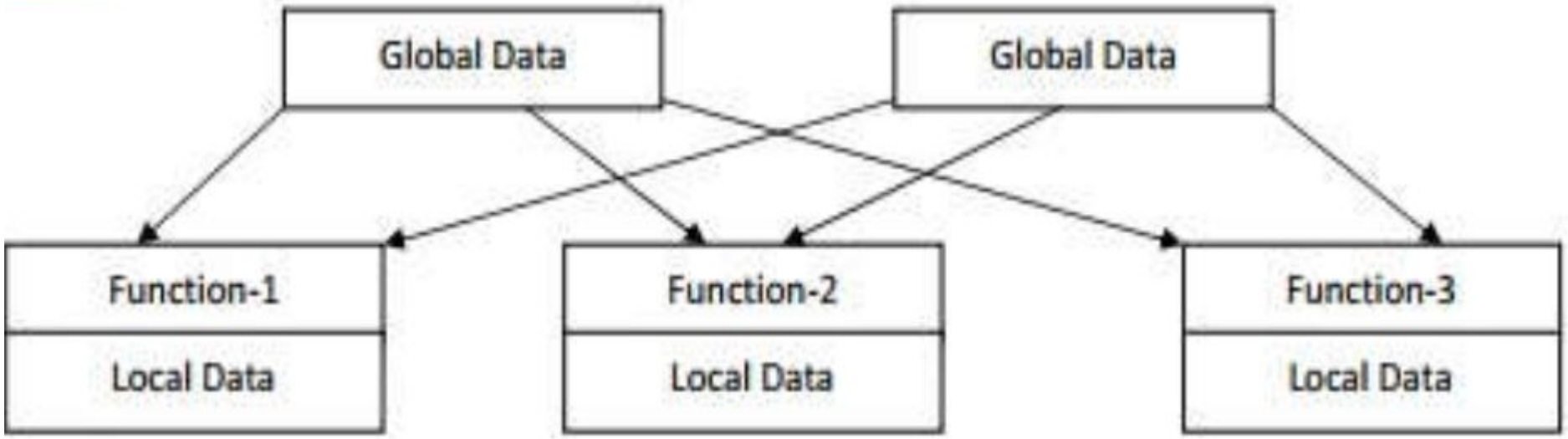
UNIT-I

- *Review of Structured Programming*
- *Structured versus Object Oriented Programming (OOP)*
- *Features of OOP*
- *Basics of C++*
- *Tokens, I/O*
- *Statements*
- *Structure of Program*
- *Operators, and Expressions*
- *Flow of Control*
- *Abstract Data Types (ADT)*
- *Variable*
- *Scope Resolution*
- *Operator*

Structured Programming

- Conventional programming, using **high level languages** such as COBOL, FORTRAN, and C, is commonly known as Structural programming .
- The problem is viewed as **a sequence of things** to be done such as reading, calculating, and printing.
- A number of **functions are written** to accomplish these tasks.
- Divide the whole program into **small modules**, so that the program becomes easy to understand.

Structured Programming



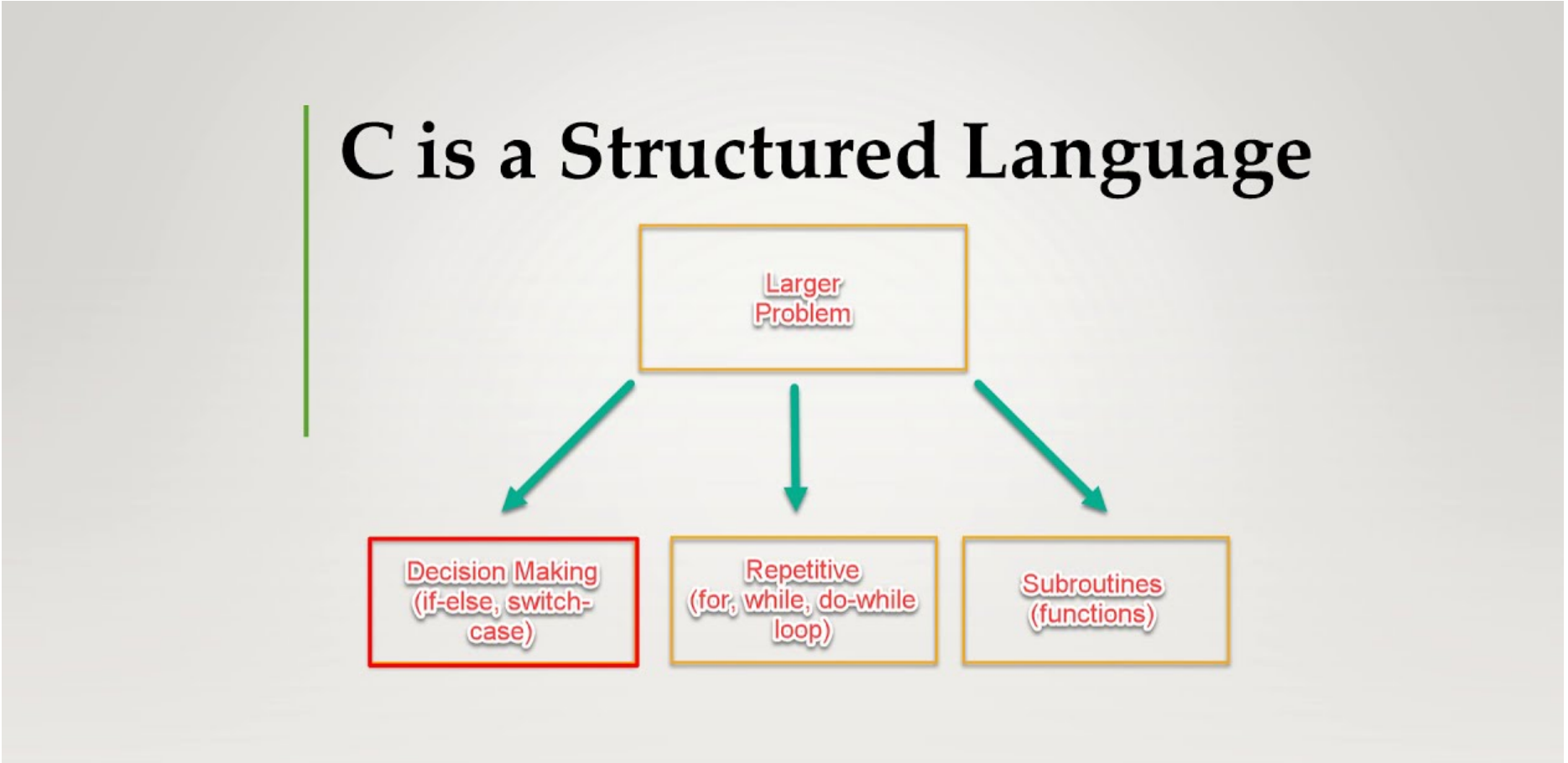
Structured Programming

- It enables the programmer to understand the **program easily**.
- Easy to **debug** the error.
- Time Efficient.

Features of Structured Programming

- Top Down Design.
- Code reusability.
- Information hiding.

Structured Programming



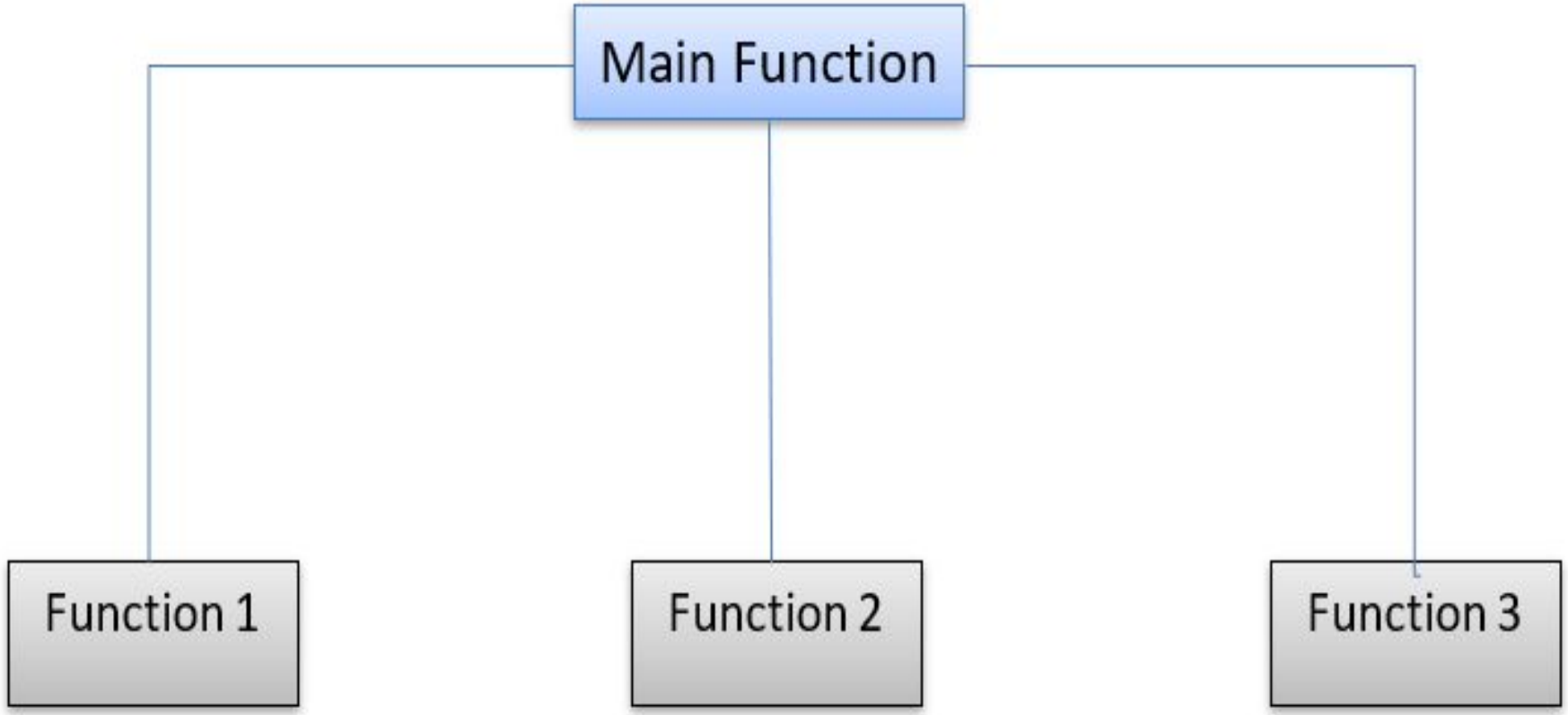
Function

- Structural Programming divide the whole program in to small blocks called functions.
- Function is a block of statements that are executed to perform a task.
- Function plays an important role in structured programming.
- Using a function is like hiring a person to do a specific job.

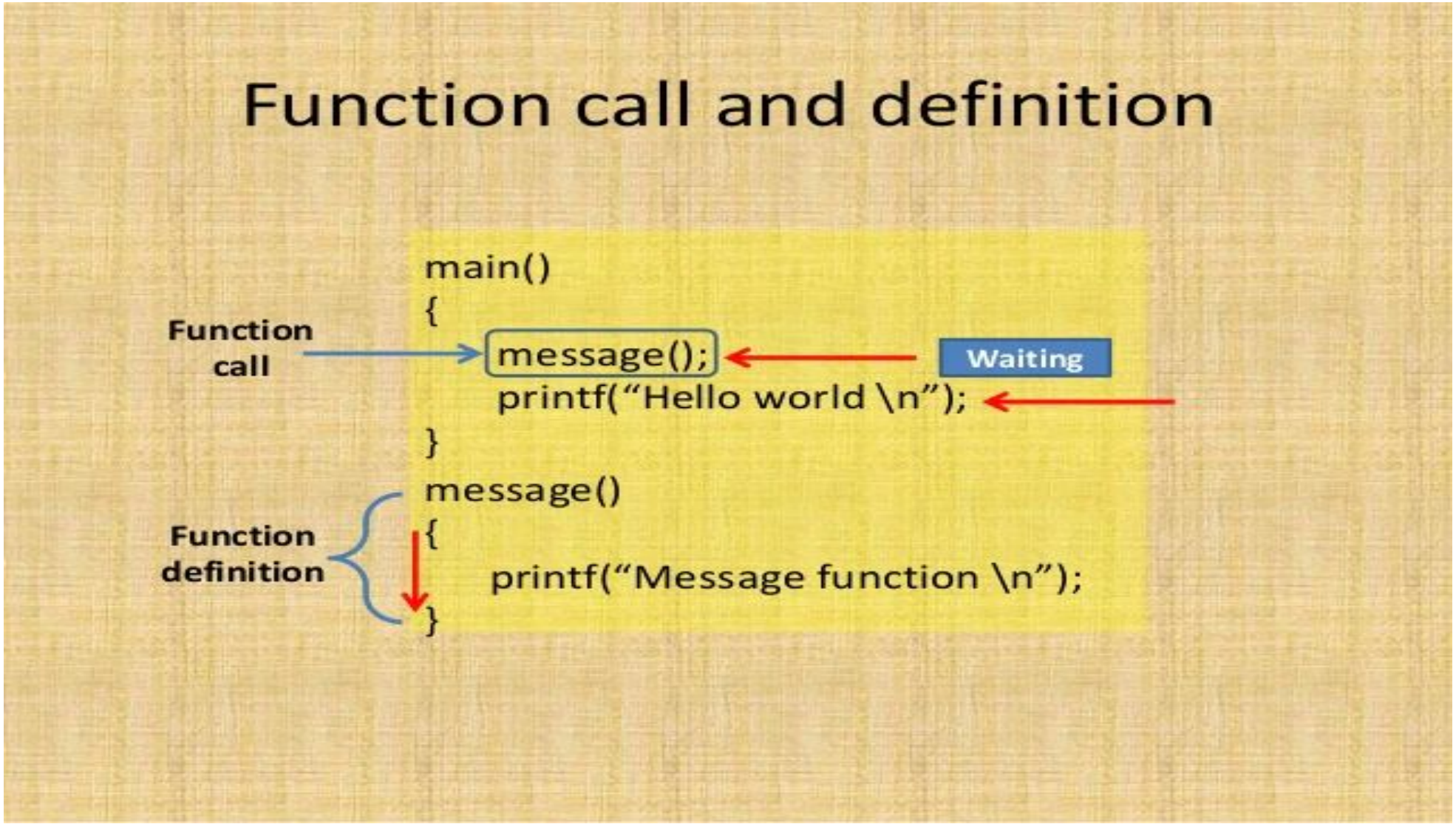
Function conti…

- When a function is called by its name the control moves to the function definition and executes all the statements written in it.
- Semi colon is used after the name of function. myfunction();
- Function Definition : It contains the instructions that are executed when a function is called
- Semi colon is not used after the name of function. void myfunction() {}

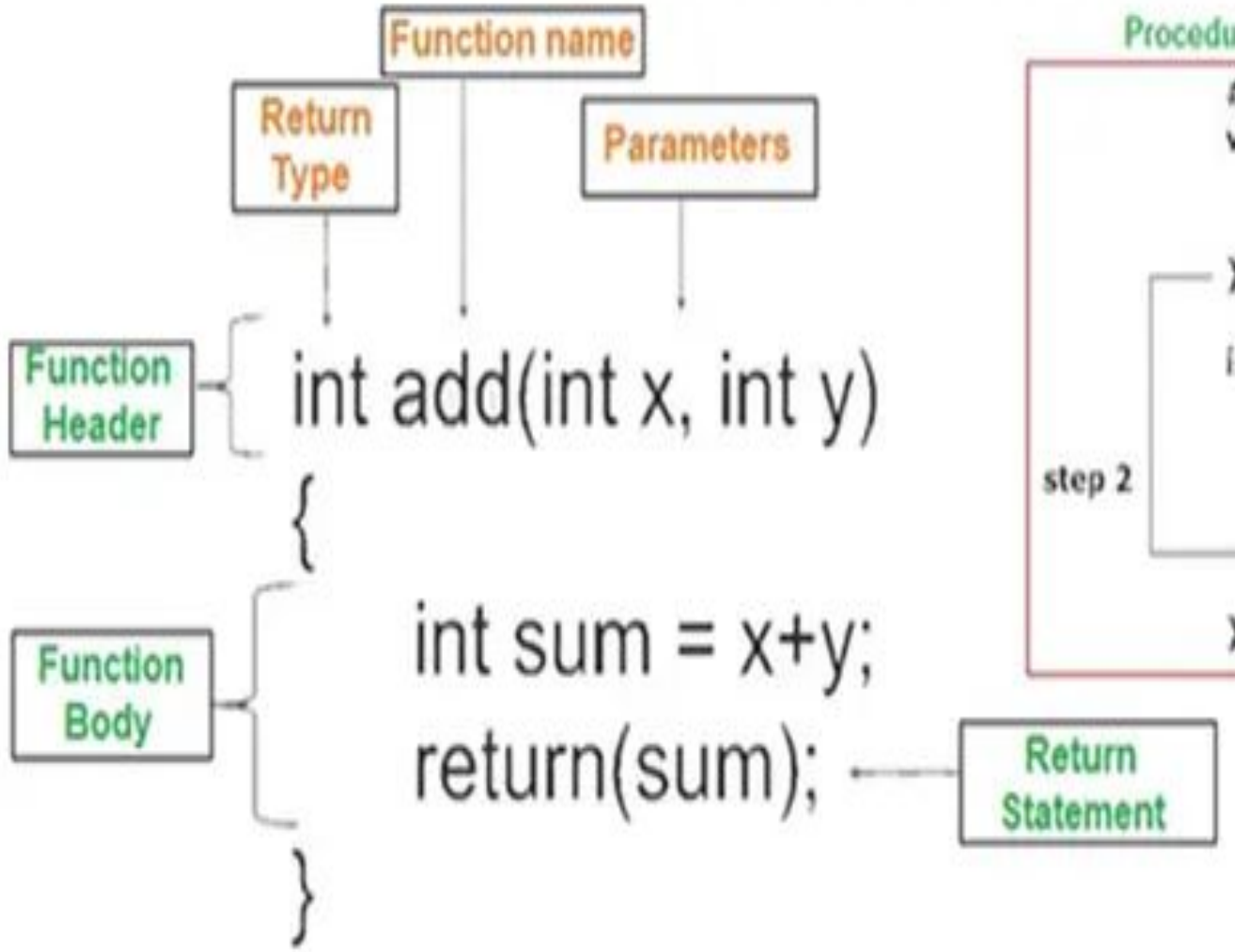
Function

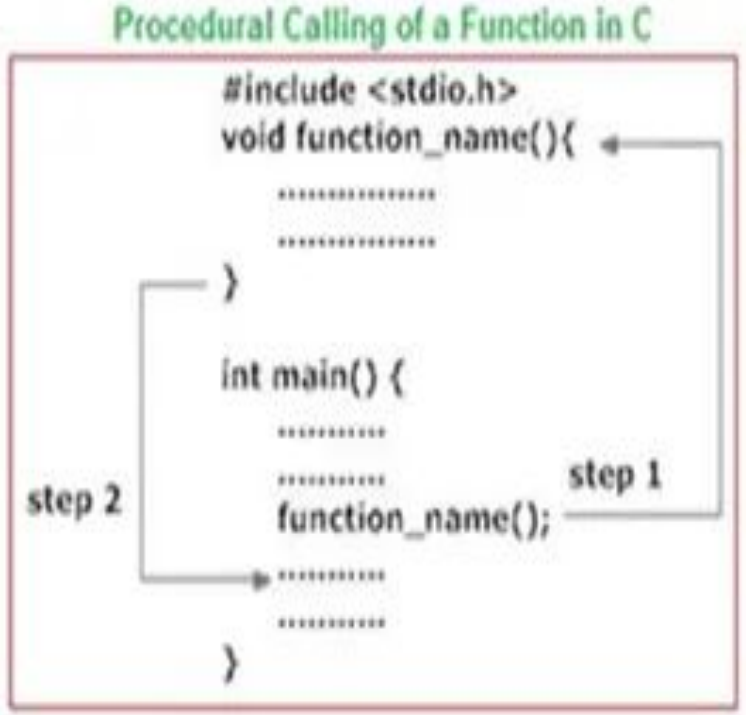


Function


Function call and definition
main()
{
message();
printf("Hello world \n");
}
message()
{
printf("Message function \n");
}
Function call
Waiting
Function definition

Function





Output ?

Object-Oriented Programming

- A rich style in program structure based on a collection of classes and object.
- Most popular current representatives are C++, and Java.
- An object-oriented design model.
- Principle programming techniques.
- Relation to other models (higher-order programming, component based programming, functional).

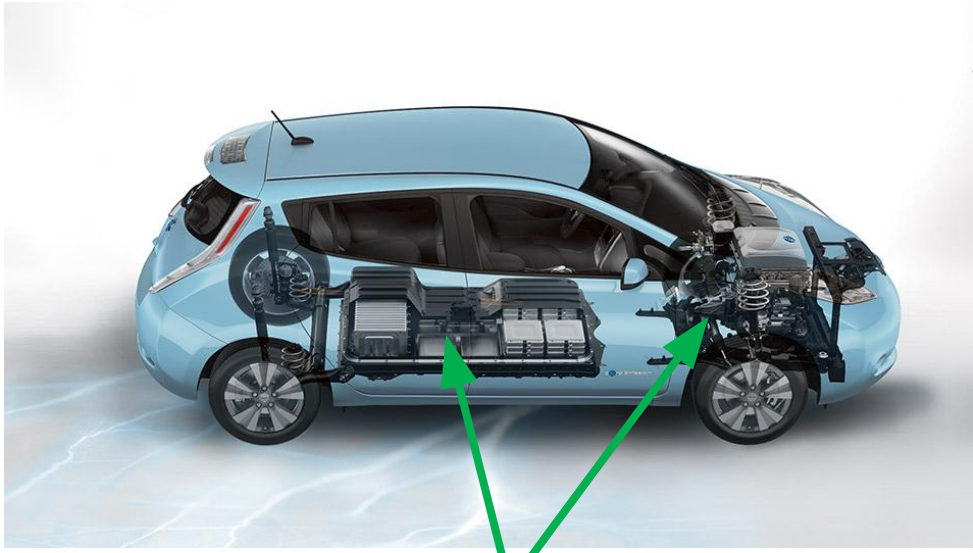
Object Oriented Programming

- OOP defines *classes* to represent these things.
- Classes can contain data and methods (internal functions).
- Classes control access to internal data and methods.
- A public interface is used by external code when using the class.
- This is a highly effective way of modeling real world problems inside of a computer program.

"Class Car"

public interface



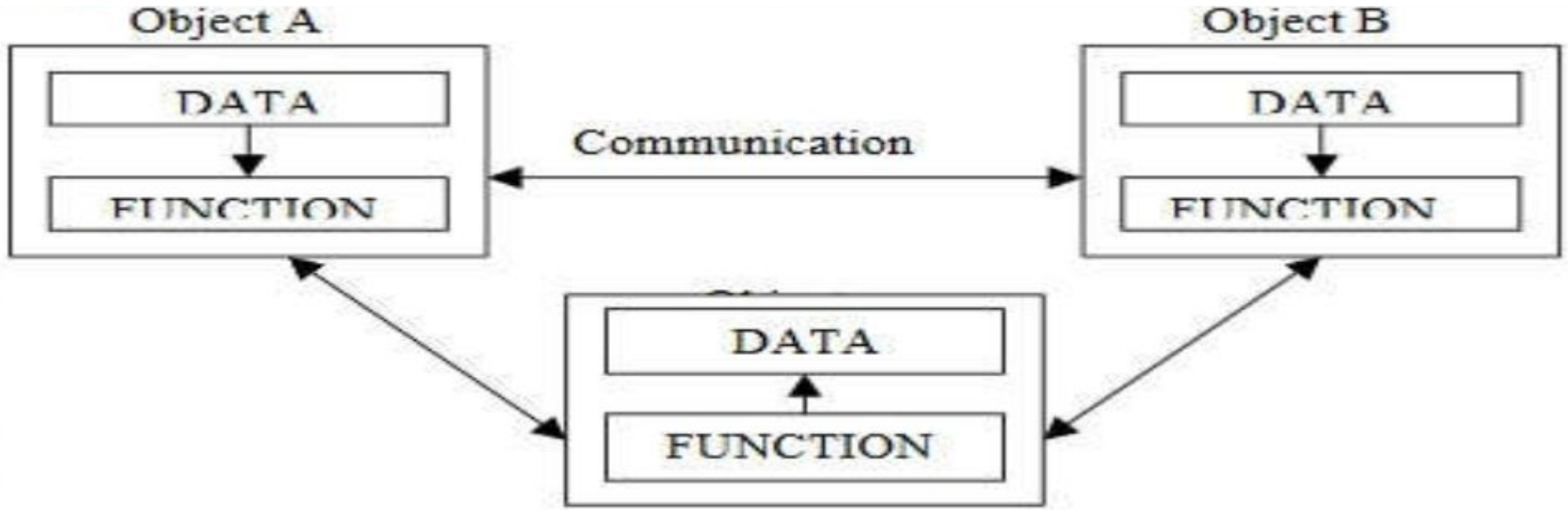


private data and methods

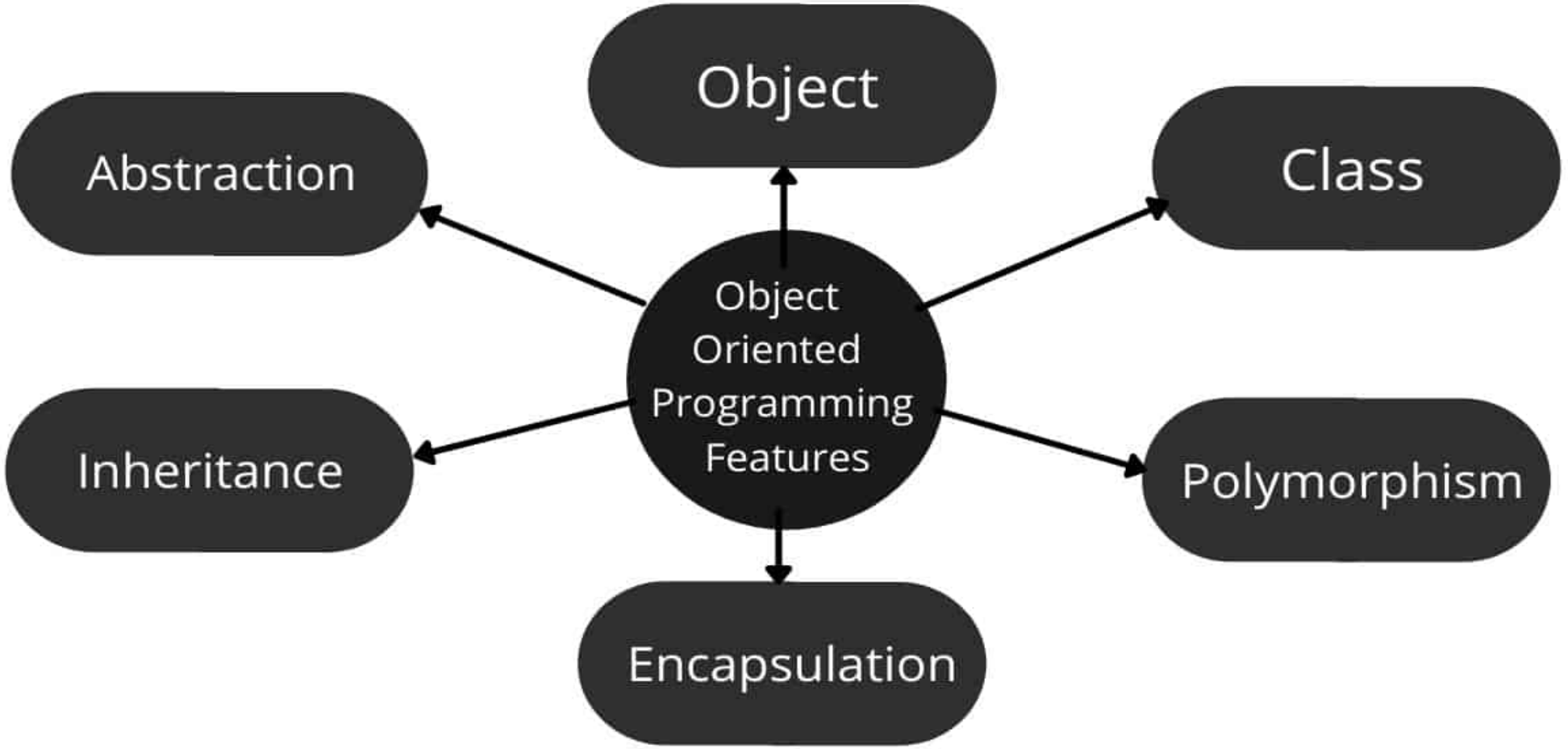
Object-Oriented Programming

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- Follows bottom-up approach in program design

Object Oriented Programming



Object Oriented Programming features



Object Oriented Programming features

- Object oriented programming concept use to make powerful, robust and secure programming instruction.
- Each program is made up of many entities called Classes.
- Objects become the fundamental units and have behaviour, or a specific purpose, associated with them.
- Objects cannot directly access another object's data.

Advantages of Object-oriented programming

- *Real world programming.*
- *Ability to simulate real-world event much more effectively*
- *Reusable of code.*
- *Information hiding.*
- *Data becomes active*
- *Better able to create GUI (graphical user interface) applications*
- *Programmers are able to reach their goals faster*
- *Programmers are able to produce faster, more accurate and better-written applications.*

Applications of Object-oriented programming

- *Real-business systems are often much more complex and contain many more objects with complicated attributes and methods.*

- *OOP is useful in these types of applications because it can simplify a complex problem. The promising areas for application of OOP include:*

- *Simulation and modelling*
- *Object-oriented databases*
- *Hypertext, hypermedia and expert Ext*
- *Al and expert systems*
- *Neural networks and parallel programming*
- *Decision support and office automation systems*
- *CIM/CAM/CAD systems*

Difference between Object Oriented Programming and Structural Programming

- *Structural programming creates a step by step program that guides the application through a sequence of instructions. While Object-Oriented programming is much more similar to the way the real world works.*
- *In Structural programming each program is made up of many small functions. While in Object-Oriented programming each program is made up of many entities called objects and class.*
- *Structural programming focuses on the idea that all algorithms are executed with functions and data that the programmer has access and Each instruction is executed in order. While in Objects become the fundamental units and have behavior, or a specific purpose, associated with them*

Difference between Object Oriented Programming and Structural Programming

S. No.	Structured Programming	Object-Oriented Programming
1	It is a subset of procedural programming.	It relies on concept of objects that contain data and code.
2	Programs are divided into small programs or functions.	Programs are divided into objects or entities.
3	It is all about facilitating creation of programs with readable code and reusable components.	It is all about creating objects that usually contain both functions and data.
4	Its main aim is to improve and increase quality, clarity, and development time of computer program.	Its main aim is to improve and increase both quality and productivity of system analysis and design.
5	It simply focuses on functions and processes that usually work on data.	It simply focuses on representing both structure and behaviour of information system into tiny or small modules that generally combines data and process both.
6	It is a method of organizing, managing and coding programs that can give or provide much easier modification and understanding.	It is a method in which set of objects can vary dynamically and can execute just by acting and reading to each other.

Difference between Object Oriented Programming and Structural Programming

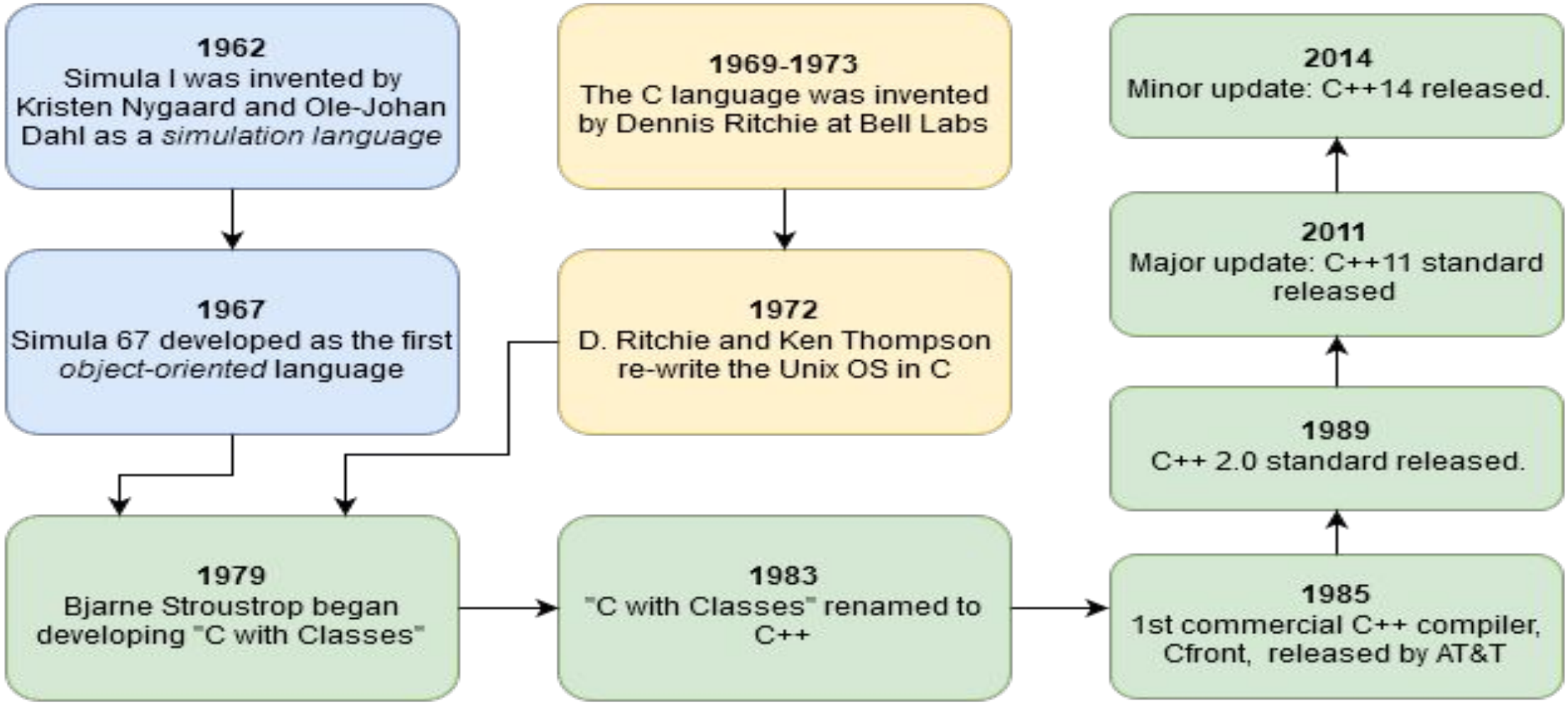
S. No.	Structured Programming	Object-Oriented Programming
7	In this, methods are written globally and code lines are processed one by one i.e., Run sequentially.	In this, method works dynamically, make calls as per need of code for certain time.
8	It generally follows “Top-Down Approach”.	It generally follows “Bottom-Up Approach”.
9	It provides less flexibility and abstraction as compared to object-oriented programming.	It provides more flexibility and abstraction as compared to structured programming.
10	It is more difficult to modify structured program and reuse code as compared to object-oriented programs.	It is less difficult to modify object-oriented programs and reuse code as compared to structured programs.
11	It gives more importance of code.	It gives more importance to data.
12	It gives less importance of security of data.	It gives less importance of security of data.

C++ Language

Exercises

1. Write a program in C++ to find the area of circle using input value ?
2. Write a program in C++ to Arithmetic operations of two integer numbers ?
3. Write a program in C++ to print the upper case letters to lower case ?
4. Write a program in C++ to show of a numeric value of a variable using hex, oct, and dec manipulator functions. ?
5. Write a program in C++ to check whether the given character upper, lower , numbr or not ?
6. Write a program in C++ to find biggest of three given numbers ?
7. Write a program in C++ to display the name of the day in a week .here given input range is 1 to 7. ?
8. Write a program in C++ to display the even or odd numbers between 1 to 100 using while loop ?
9. Write a program in C++ to print a word form of given number between 0 to 9 using Switch ?
10. Write a program in C++ to find the smallest of 10 numbers ?

History of C++


1962
Simula I was invented by Kristen Nygaard and Ole-Johan Dahl as a *simulation language*
1967
Simula 67 developed as the first *object-oriented* language
1969-1973
The C language was invented by Dennis Ritchie at Bell Labs
1972
D. Ritchie and Ken Thompson re-write the Unix OS in C
1979
Bjarne Stroustrop began developing "C with Classes"
1983
"C with Classes" renamed to C++
1985
1st commercial C++ compiler, Cfront, released by AT&T
1989
C++ 2.0 standard released.
2011
Major update: C++11 standard released
2014
Minor update: C++14 released.

Characteristics of C++

"Actually I made up the term 'object-oriented', and I can tell you I did not have C++ in mind."
– Alan Kay (helped invent OO programming, the Smalltalk language, and the GUI)

- C++ is a
  - Compiled.
    - A separate program, the compiler, is used to turn C++ source code into a form directly executed by the CPU.
  - Strongly typed and unsafe
    - Conversions between variable types must be made by the programmer (strong typing) but can be circumvented when needed (unsafe)
  - C compatible
    - call C libraries directly and C code is nearly 100% valid C++ code.
  - Capable of very high performance
    - The programmer has a very large amount of control over the program execution
  - Object oriented
    - With support for many programming styles (procedural, functional, etc.)
- No automatic memory management
  - The programmer is in control of memory usage

Difference b/w c and c++

C Language	C++ Language
Procedural Programming Language	Object Oriented Programming Language
Headerfile : #include<stdio.h>	#include<iostream.h>
"\n" is used to go to the next line	we can use endl statement
Local variables declared only the start of a C program	It can be declared anywhere in a program, before they are used
Return type for a function is optional	Return type must be specified
Do not permit data Hiding	They permit data hiding
Bydefault structure members are public	class members are private
We can call a main() function within a program	This is not allowed

BASIC TERMINOLOGY of C++

object - An object is an instance of a class which combines both data and functions together.

Method - an action performed by an object (a verb)

Attribute - description of objects in a class

class - a category of similar objects

- does not hold any values of the object's attributes

By default all members declared inside a class are private to that class

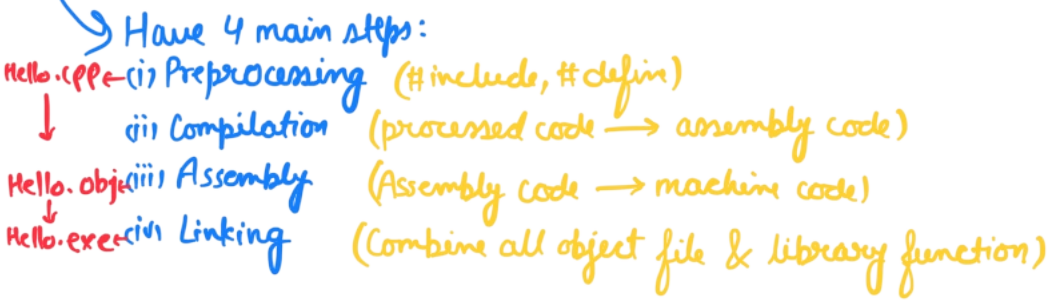
Basic of C++

1. Compilation
2. The # Preprocessor
3. Declarations and Definitions
4. main () function
5. Comments
6. Class Variables & Functions
7. Input, output operators
8. Cascading I/O operators
9. Control Statement

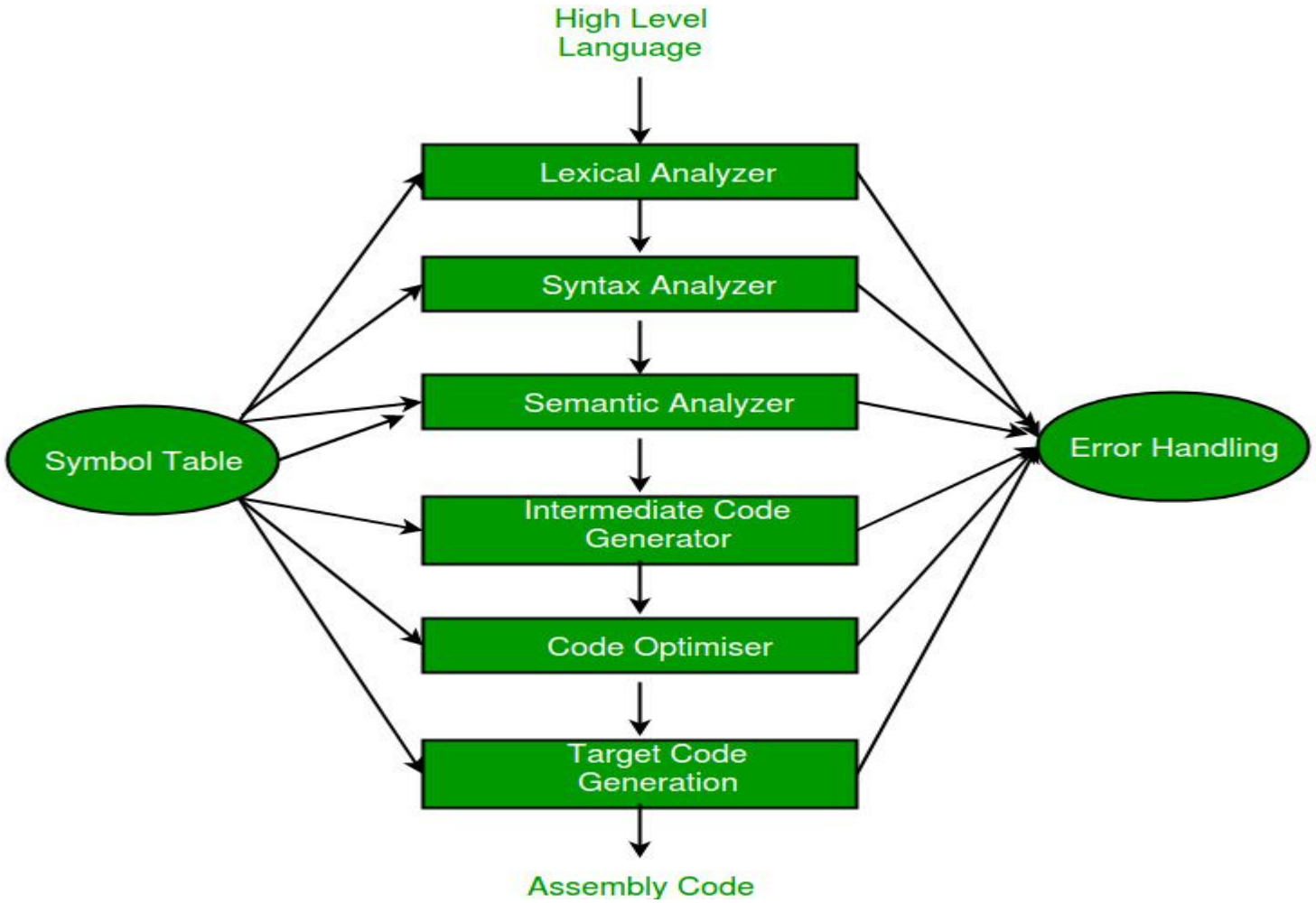
1. Compilation

- C++ programs can range from a handful of statements to hundreds of thousands.

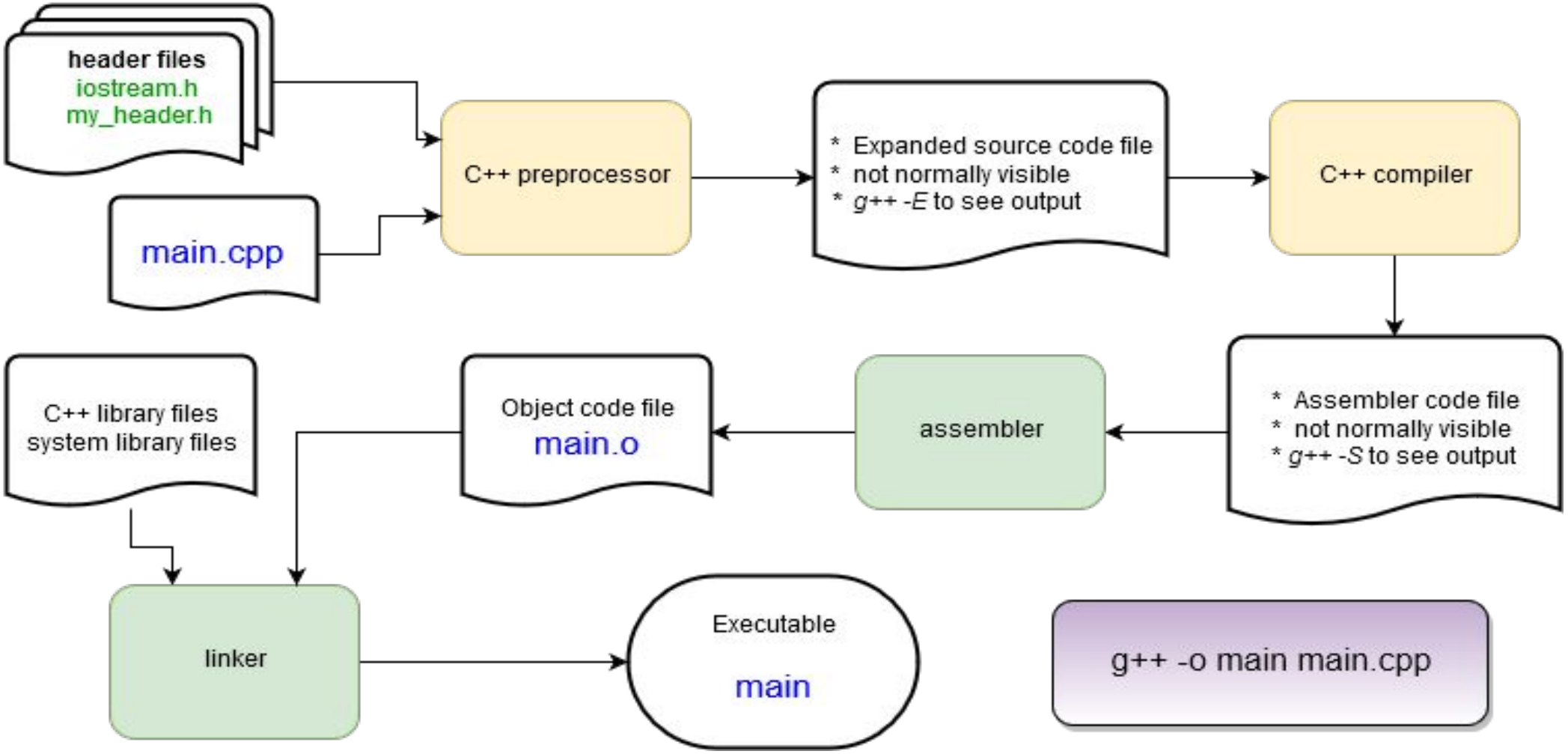
- May be written by one person or by a team



1. Compilation



Behind the Scenes: The Compilation Process

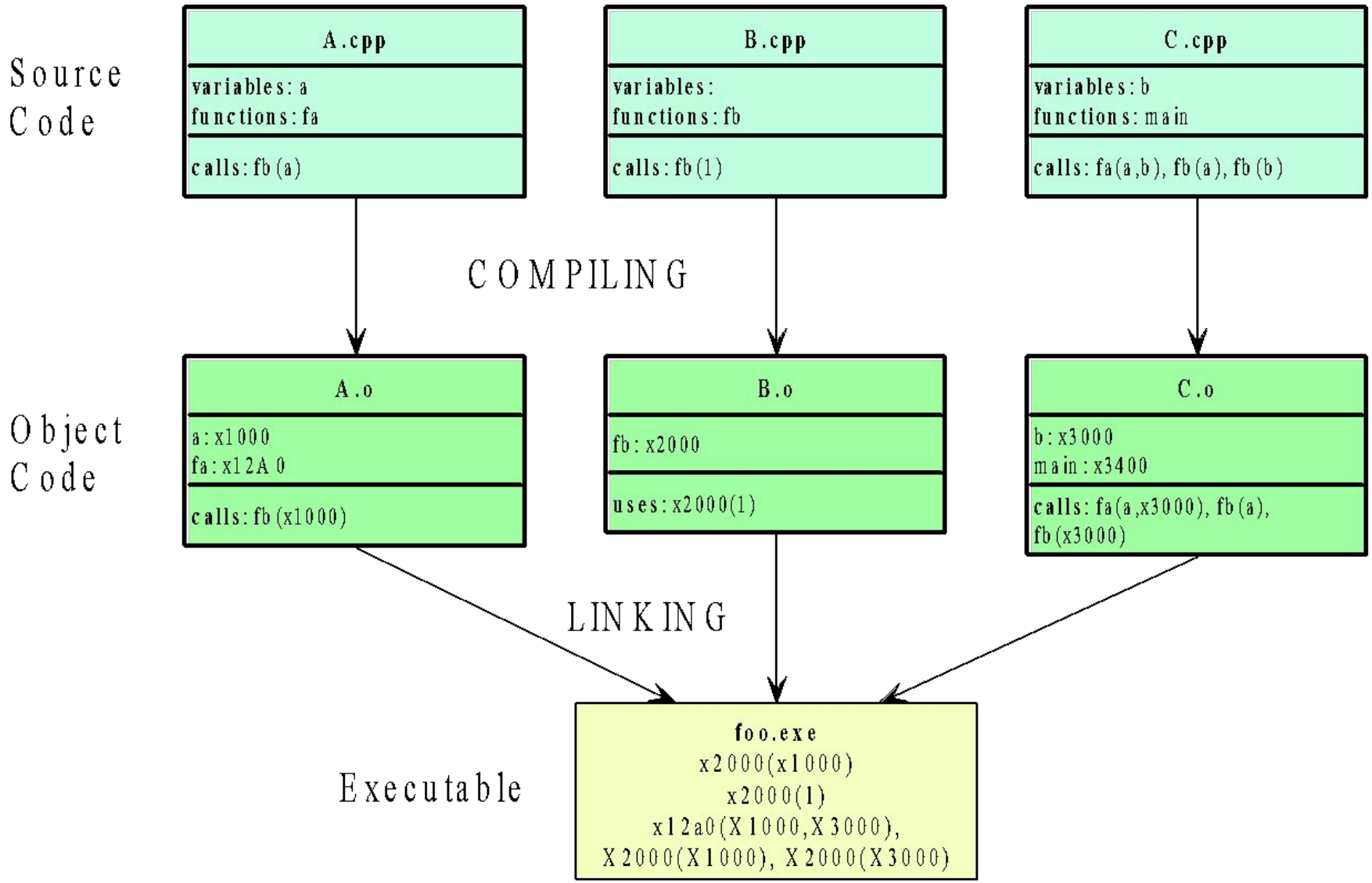


Multiple File C++ Programs

- Team members can work in parallel on separate files.
- Files are compiled separately
  - each individual compilation is fast
- Separately compiled code is linked to produce the executable
  - linking is much faster than compilation

Separate Compilation

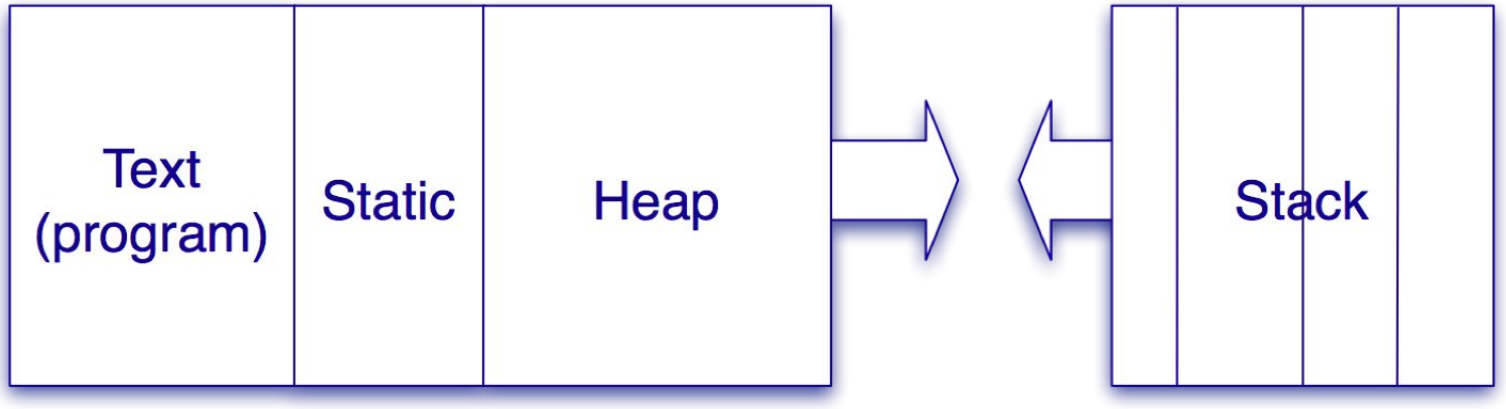
- Each file of source code
  - programming language text
- is compiled to produce a file of object code
  - binary code, almost executable
  - exact addresses of variables and functions not known, represented by symbols
- All object code files linked to produce the executable
  - Linking mainly consists of replacing symbols by real addresses.


Source Code
A.cpp
variables: a
functions: fa
calls: fb(a)
B.cpp
variables:
functions: fb
calls: fb(1)
C.cpp
variables: b
functions: main
calls: fa(a,b), fb(a), fb(b)
COMPILING
Object Code
A.o
a: x1000
fa: x12A0
calls: fb(x1000)
B.o
fb: x2000
uses: x2000(1)
C.o
b: x3000
main: x3400
calls: fa(a,x3000), fb(a), fb(x3000)
LINKING
Executable
foo.exe
x2000(x1000)
x2000(1)
x12a0(X1000,X3000),
X2000(X1000), X2000(X3000)

Memory Layout

The address space consists of (at least):

Text:	executable program text (not writable)
Static:	static data
Heap:	dynamically allocated global memory (grows upward)
Stack:	local memory for function calls (grows downward)



2. The # Preprocessor

- The preprocessor runs before the compiler proper
  - modifies the source code
  - processes preprocessor instructions
    - lines beginning with #
  - strips out comments

Processor Instructions

- #include
  - insert a file
- #define
  - define a macro
- #ifdef, #ifndef, #endif
  - check to see if a macro has been defined

#include

- Inserts a file or header into the current source code.
- Two versions
  - #insert <headerName>
    - inserts a system header file from a location defined when the compiler was installed
  - #insert “fileName”
    - inserts a file from the current directory

Header Files

- C++ (along with C) uses *header files* as to hold definitions for the compiler to use while compiling.
- A source file (file.cpp) contains the code that is compiled into an object file (file.o).
- The header (file.h) is used to tell the compiler what to expect when it assembles the program in the linking stage from the object files.
- Source files and header files can refer to any number of other header files.

C++ language headers aren't referred to with the .h suffix. <iostream> provides definitions for I/O functions, including the *cout and cin* function.

```
#include <iostream>

using namespace std;

int main()
{
    string hello = "Hello";
    string world = "world!";
    string msg = hello + " " + world ;
    cout <<  msg << endl;
    msg[0] = 'h';
    cout <<  msg << endl;
    return 0;
}
```

#define

- Used to define macros (symbols that the preprocessor will later substitute for)
- Sometimes used to supply constants

```
#define VersionNumber "1.0Beta1"
int main()
{
cout << "Running version “<< VersionNumber<< endl;
}
```

- Much more elaborate macros are possible, including ones with parameters

Macro Example

```
#include <iostream>
using namespace std;

// Define a constant for the value of PI
#define PI 3.14159

int main()
{
    double radius = 4.0;


    double area = PI * radius * radius;

    cout << "Area of circle with radius " << radius << " is " << area;

    return 0;
}
```

Macro Example

```
#include <iostream>
using namespace std;
#define PRINT(x) cout << "Value is: " << x

int main()
{
    int value = 42;
    PRINT(value);
    return 0;
}
```

Macro Example

```
#include <iostream>
#define PRINT_RECTANGLE(width, height)
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
            {
            std::cout << "*";
             }
         std::cout << std::endl;
    }

int main()
{

PRINT_RECTANGLE(4, 3);
    return 0;
}
```

3. Declarations and Definitions

- Some of the most common error messages are
  - … is undeclared
  - … is undefined
  - … is defined multiple times

- Fixing these requires that you understand the difference between declarations and definitions
  - and how they relate to the program structure

Declarations

- A declaration in C++
  - introduces a name for something
  - tells what “kind” of thing it is
  - gives programmers enough information to use it

Definitions

- A definition in C++
  - introduces or repeats a name for something
  - tells what “kind” of thing it is
  - tells what value it has and/or how it works
  - gives the compiler enough information to generate this and assign it an address

4. main() Function

main.cpp

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main()
6   {
7       cout << "Hello world!" << endl;
8       return 0;
9   }
10
```

The *main* routine – the start of **every** C++ program! It returns an integer value to the operating system and (in this case) takes no arguments: main()

The **return** statement returns an integer value to the operating system after completion. 0 means "no error". C++ programs **must** return an integer value.

loads a *header* file containing function and class definitions

Loads a *namespace* called *std*. Namespaces are used to separate sections of code for programmer convenience. To save typing we'll always use this line in this tutorial.

main.cpp

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main()
6   {
7       cout << "Hello world!" << endl;
8       return 0;
9   }
10
```

- *cout* is the *object* that writes to the stdout device, i.e. the console window.
- It is part of the C++ standard library.
- Without the "using namespace std;" line this would have been called as *std::cout*. It is defined in the *iostream* header file.
- << is the C++ *insertion operator*. It is used to pass characters from the right to the object on the left. *endl* is the C++ newline character.

Comments

- C++ syntax is very similar to C, Java, or C#.

- Curly braces are used to denote a code block (like the main() function):

{ … some code … }

- Statements end with a semicolon:

- Comments are marked for a single line with a // or for multilines with a pair of /* and */ :

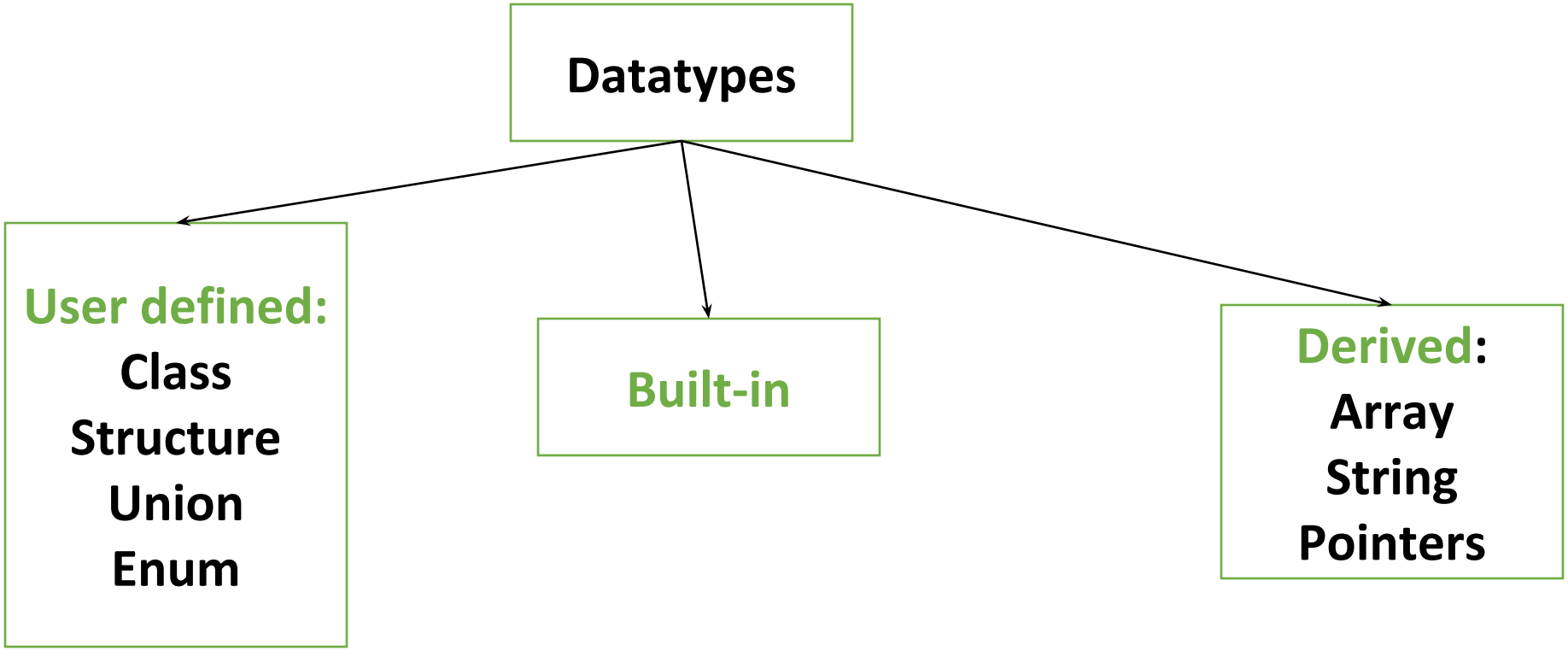
```
// this is a comment.
/* everything in here
        is a comment */
```

- Variables can be declared at any time in a code block.

Tokens, Keywords and identifiers

- Tokens , a smallest individual units.
  - Keywords, identifiers, constants,..etc
- Keywords implements specific c++ language features
- Identifiers refers the names of variables, functions, arrays..etc

Data types


Datatypes
User defined:
Class
Structure
Union
Enum
Built-in
Derived:
Array
String
Pointers

Structure

- Structure is the collection of variables of different types under a single name for better visualization of problem.
- Arrays is also collection of data but arrays can hold data of only one type whereas structure can hold data of one or more types.

```
struct person
{
 char name[50];
int age;
float salary;
 };
```

How to define a structure variable?

```
person bill;
```

How to access members of a structure?

```
bill.age = 50;
```

Struct Example

```
#include <iostream.h>
struct person
{
char name[50];
 int age;
float salary;
 };

 int main()
{
 person p1;
 cout << "Enter Full name: ";
 cin>>p1.name;
 cout << "Enter age: ";
 cin >> p1.age;
 cout << "Enter salary: ";
 cin >> p1.salary;
 cout << "\nDisplaying Information." << endl;
 cout << "Name: " << p1.name << endl;
 cout <<"Age: " << p1.age << endl;
 cout << "Salary: " << p1.salary;
 return 0;
 }
```

Union

variables कितने भी हो, store करने का ठिकाना एक ही है, अब जो चाहे store करो।

Ex: union &x { int x; float y; }; &a;
a.x = 5 → ठिकाने में int x = 5 store हो गया।
a.y = 4.5 → ठिकाने से int गायब। अब float store हो गया। → अब अगर int print करोगे तो garbage value मिलेगी।

- Unions in C++ is a user defined data type that uses the same memory as other objects from a list of objects. At an instance it contains only a single object.
- Example

```
#include <iostream.h>

union Emp
 {
 int num;
 double sal;
 };

 int main()
 {
 Emp value;
 value.num = 2;
 cout << "Employee Number::" << value.num
 << "\nSalary is:: " << value.sal << endl;
 value.sal = 2000.0;
 cout << "Employee Number::" << value.num
 << "\nSalary is:: " << value.sal << endl;
 return 0;
 }
```

Ouput

```
Employee number is::2
Salary is::0
Employee number is::0
Salary is::2000
```

Union

output : 8

Union होता सबसे Largest storage value लेता है।

- Example

```
union test
{
    int x;
    char arr[8];
    int y;
};
int main()
{
    cout<< sizeof(union test) <<endl;
    return 0;
}
```

Union

- Example

```
union test
{
    int x;
    char arr[8];
    int y;
};
int main()
{
    cout<< sizeof(union test) <<endl;
    return 0;
}
```

enum

Used to assign name to integer constants.

- Enum, which is also known as enumeration is a user-defined data type that consists of integral constants.
- It provides a way to define and group integral constants.
- To define an enumeration, keyword enum is used.
- Enums are generally used when you expect the variable to select one value from the possible set of values.

enum

```
#include <iostream>
using namespace std;
// Defining enum Year
enum year {Jan=0, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};
int main()
{
    int i;
for (i = Jan; i <= Dec; i++)
        cout << i << " ";
    return 0;
}
```

Output

0 1 2 3 4 5 6 7 8 9 10 11

enum

```
#include <iostream>
using namespace std;
enum colors{red=5, black};
enum suit{heart, diamond=8, spade=3, club};
int main() {
   cout <<"The value of enum color : "<<red<<","<<black;
   cout <<"\nThe default value of enum suit :
"<<heart<<","<<diamond<<","<<spade<<","<<club;
   return 0;
}
```

Output

The value of enum color : 5,6
The default value of enum suit : 0,8,3,4

enum

```
#include <iostream>
    using namespace std;
    enum  cat
    {
        temp = 7, tempp, temppp;
    };
    int main()
    {
        float div = 18;
        div /= tempppp;
        cout << div << endl;
        return 0;
    }
```

Output

2

BUILT-IN Datatypes

Data Type	Size	Range
char	1 byte	-127 to 127 or 0 to 255
int	4 bytes	-2147483648 to 2147483647
unsigned int	4 bytes	0 to 4294967295
signed int	4 bytes	-2147483648 to 2147483647
short int	2 bytes	-32768 to 32767
long int	8 bytes	-2,147,483,648 to 2,147,483,647
long long int	16 bytes	-(2^63) to (2^63)-1
float	4 bytes	
double	8 bytes	
long double	16 bytes	
Bool	1 bytes	

Array

- Array is a collection of elements of same type referred using a unique name.
- Each element of an array are stored in memory locations indexed from 0 through n number of its elements.
- The lowest indexed will be the first element and highest indexed the last element of an array.
- type array_name[array_size_1]
- int age[5];

Example

```
#include <iostream.h>
  void main()
  {
  int i;
  float mark[6];
  cout << "Enter the marks of your 6 subjects:: \n";
  for(i=0; i<6; i++)
  {
  cin >> mark[i];
  }
  float sum=0;
for(i=0;i<6;i++)
  {
  sum += mark[i];
  }
  float ave = sum/6;
  cout << "Average Marks is::" << ave << '\n';
  }
```

Strings

- C++ provides convenient and powerful tools to manipulate strings.

- There are two types of strings commonly use in C++ programming language:

1. Strings that are objects of string class (The Standard C++ Library string class)

2. C-strings (C-style Strings)

Strings

- C++ provides convenient and powerful tools to manipulate strings.

- There are two types of strings commonly use in C++ programming language:

1. Strings that are objects of string class (The Standard C++ Library string class)

2. C-strings (C-style Strings)

C Style strings in C++

- char str[] = "C++";
- char str[4] = "C++";
- char str[] = {'C','+','+','\0'};
- char str[4] = {'C','+','+','\0'};

Example

```
#include <iostream>
#include<string>
using namespace std;
int main()
{
char str[100];
cout<<"Enter a string: ";
cin>>str;
cout<<"You entered: "<<str<<endl;
cout<<"\nEnter another string: ";
cin>>str;
cout<<"You entered: "<<str<<endl;
return 0;
}
```

1st Cin → Hello world
1st print → Hello
2nd Cin → ले ही नहीं पाएँगा।
2nd print → World

This is because already buffer में world था जो cin के आने से array में Hello को replace कर दिया।

space read नहीं करेगा।

Example

```
#include <iostream>
using namespace std;

int main()
{
  string S = “Hello World";
  cout << "Your string is= ";
  cout << S << endl;
  return 0;
}
```

output → Your string is Hello World

C++ strings

Formatted Input: Stream extraction operator

```
cin >> stringObject
```

Unformatted Input: getline function for a string

```
getline( cin, s)
//reads an entire line of characters into s
```

string is not a pointer, but a class

Example

```
#include <iostream>
using namespace std;

int main()
{
  string s;
  cout<<"Enter String"<<endl;
  cin>>s;
  cout<<"String is: "<<s<<endl;
  return 0;
}
```

Example

```
#include <iostream>
using namespace std;
int main()
{
  string s;
  cout << "Enter String" << endl;
  getline(cin, s);
  cout << "String is: " << s << endl;
  return 0;
}
```

THANKS…..